

## Final Project Description

---

### Final Project Options

**Projects can be performed either alone or by a team of two people.**

In addition to the default project, students also have the option to do another project. If choosing to do another project, students need to get in touch with the instructor about it and send a project proposal for review. The project itself needs to be challenging enough and involves both OpenMP and MPI.

---

### Computing Resources

You should run all experiments on the ISAAC clusters:

- **ISAAC-NG cluster:** Use either the *short* or *long* QoS and partition, with your default campus account (acf-utk0011).
- **ISAAC-ORI cluster:** If you need additional resources, you may also use ISAAC-ORI. For this system, submit jobs with qos=condo and partition=condo-cosc462, using the class project account (ISAAC-UTK0414).

Important: Do not run jobs on the login node. All computations must be submitted as batch jobs through the scheduler.

---

### Default Final Project Problem Introduction

MapReduce is a simple but powerful programming model for processing large datasets in parallel. It has two main stages:

1. Map stage – The input dataset is represented as a collection of tuples <key, value>. Each mapper processes its portion of the input and groups records by their key, producing intermediate “buckets” (equivalence classes). For example, all records with the same key end up in the same bucket.
2. Reduce stage – Each reducer processes one of these buckets and combines all the values associated with a given key into a single result (e.g., by summing, counting, or averaging).

Although popularized by Google, MapReduce is based on an older parallel programming idea and is surprisingly general.

## Example: Word Count

The word count problem illustrates MapReduce well. Suppose the input is a large text. The goal is to output how many times each word appears.

- Map step: Each word in the text is converted into a tuple  $\langle \text{word}, 1 \rangle$ . For instance, the word *cat* produces  $\langle \text{cat}, 1 \rangle$ .
- Shuffle step (between map and reduce): All tuples with the same word are grouped together, e.g.,  $\langle \text{cat}, 1 \rangle$ ,  $\langle \text{cat}, 1 \rangle$ ,  $\langle \text{cat}, 1 \rangle$  go into the *cat* bucket.
- Reduce step: The reducer adds up the counts in each bucket, producing  $\langle \text{cat}, 3 \rangle$ ,  $\langle \text{dog}, 2 \rangle$ , and so on.

## Optimizations in MapReduce

1. Combiners (local aggregation):  
Instead of sending all  $\langle \text{word}, 1 \rangle$  tuples across the network, each mapper can first combine its own results locally. For example, if a mapper sees the word *cat* 50 times, it sends  $\langle \text{cat}, 50 \rangle$  instead of 50 separate  $\langle \text{cat}, 1 \rangle$  messages. This greatly reduces communication cost.
2. Batching & partitioning:  
To further improve efficiency, multiple key groups (e.g., *cat*, *dog*, *test*, *homework*) can be sent together in a single message to a reducer. A hash function  $H(\text{key})$  decides which reducer handles each word. If there are  $R$  reducers, the function maps each key to one of the  $R$  reducers ( $0 \leq H(\text{key}) \leq R-1$ ). This balances the workload across reducers and minimizes communication overhead.

---

## What We Will Program

We will program two map-reduce programs. The project will be done in three steps:

1. The first program is an OpenMP version for wordcount map reduce on different cores in a shared memory node.
2. The second program is an MPI-OpenMP hybrid version that uses the OpenMP version to perform node-local computation and uses MPI for cross-node distributed memory computations for wordcount map reduce.

### 3. Final turn-in.

---

#### Input Text Data

The input text data are provided along with the final project.

---

#### The First Program: OpenMP Version of MapReduce (Shared Memory Cores on Single Node)

The first program asks you to write a **MapReduce pipeline** using OpenMP threads on a single node. The idea is to split work across different types of threads, each responsible for one stage of the pipeline.

##### 1. Types of Threads

- **Reader threads**
  - Read data from input files (or generate it if you're self-initializing).
  - Place each data item into a **work queue**.
  - For **word count**, each item is a single word.
- **Mapper threads**
  - Run in parallel with Readers (until all Readers finish).
  - Take items from the Reader queue and create **combined records**.
  - Example: If the word “cat” appears 2045 times, the mapper outputs <“cat”, 2045>.
- **Reducer threads**
  - Collect records produced by the Mappers.
  - Combine all partial results into one final result per key.
  - Example: If multiple Mappers report <“cat”, count>, the Reducer assigned to “cat” will sum all counts and produce the final total.
  - Each unique key should be handled by exactly one Reducer.
- **Writer threads**

- Take the final reduced results and write them to output files.

**Note:** You don't need to literally create separate OpenMP thread teams for each role. Instead, you can use work queues where threads pick up tasks. This means you can potentially use OpenMP task to achieve it, but you can also implement custom queues. Both options are valid. For example, Mapper and Reducer threads could run at different phases but reuse the same underlying OpenMP thread pool.

## 2. Work Queues & Hashing

- Each Reducer thread should have its own **queue**.
- Mapper threads place results into the appropriate Reducer's queue.
- Use a hash function  $H(\text{key})$  to decide which Reducer handles a given key.
  - Example:  $g = H(\text{key})$  where  $0 \leq g < R$ , and  $R$  is the total number of Reducers.
  - For load balance, you may set  $R = k \times \text{numMappers}$  for some constant  $k$ .

## 3. Synchronization

- Ensure that Mappers wait until Reducers finish producing input. Otherwise, Mappers may run out of work prematurely.
- Reducers should only complete once all Mapper work has been consumed and processed.
- At the end:
  - Reducers write results to files.
  - Each thread notifies the master thread that it has completed.
  - Once all work is done, the program can safely terminate.

In the second program, you will extend your OpenMP MapReduce program to run across multiple nodes using MPI. Each node will still run the OpenMP code for local computation, but nodes will also need to exchange intermediate results with one another using MPI.

The key idea is:

- **Within a node**, OpenMP handles parallel computation (Reader, Mapper, Reducer, Writer).
- **Across nodes**, MPI handles communication so that reducers get all the data they need, even when the data was generated by mappers on different nodes.

### Changes from the pure OpenMP version:

#### 1. Mapper output handling

- Instead of putting results directly into a reducer's local work queue, mapper threads should place their results into a buffer/list that is prepared for communication to other nodes.

#### 2. Sending intermediate data

- Each node should have a sender thread (or dedicated routine) that collects mapper output, groups it by destination node, and sends it to the correct node via MPI.

#### 3. Receiving intermediate data

- Each node should also have a receiver thread (or routine) that listens for incoming messages from other nodes.
- Received data should be placed into the appropriate reducer work queue on that node.

#### 4. File distribution

- Each node must process a portion of the total input files. If we assign files statically, some nodes might end up with large files and others with small files, creating imbalance.
- To avoid this, implement a master/worker scheme:
  - A master node keeps track of which files remain unprocessed.
  - Worker nodes request files from the master.

- The master responds with either the next filename or an “all done” message when no files remain.

---

## Performance Data and Tuning

You should collect and analyze performance data to understand both bottlenecks and scalability of your implementation. At a minimum, include the following:

1. Bottlenecks – Measure where the code spends time (e.g., Mapper threads waiting for Reader threads, I/O vs. mapping time, waiting for I/O, etc.).
2. Load imbalance within a node – Quantify variation in execution time across threads on the same node.
3. Load imbalance across nodes – Compare the time when the first node is ready to send data with when the last node is ready.
4. Effect of Reader threads – Experiment with different numbers of Reader threads and analyze performance impact.

---

## Step Deliverables

1. OpenMP version:
  - Source code
2. MPI-OpenMP hybrid version:
  - Source code
3. Final Comprehensive Report:

Submit a paper (8 to 12 pages in length in PDF. Prepared in LaTeX/Overleaf preferred, but Word is acceptable) that includes:

  - Introduction – Problem description, major performance bottlenecks, and overall strategy.
  - Approach – Detailed explanation of your OpenMP and hybrid MPI+OpenMP implementation approach.
  - Results & Discussion – Present and analyze performance data. Include the following:
    1. Performance Scaling:

- Speedups and efficiencies for 2, 4, 8, and 16 cores (OpenMP).
- Speedups and efficiencies for 1, 2, and 4 nodes (16 cores per node, hybrid MPI+OpenMP).
- Karp-Flatt analysis of results.
- Iso-efficiency analysis of your program.

2. Thread Scaling:

- Curves showing performance vs. number of Reader threads.
- Curves showing performance vs. number of Mapper and Reducer threads.

3. MapReduce Breakdown:

- Performance of each stage (map, reduce) as well as end-to-end performance.

4. Baseline Serial Performance:

- Use a single thread as your baseline reference point.

5. Node Scaling:

- Performance across multiple nodes with speedup and efficiency metrics.

6. Interpretation:

- Explain why you observed the speedups you achieved (both positive and negative).
- Conclusion – Reflect on what worked well and what you would change with more time.

---

### **Additional Notes**

I may schedule a short meeting during dead week for each group to demonstrate your work. Be prepared to give a short presentation of your work.

---

### **Grading breakdown:**

- 40% – Working parallel implementation with performance measurements
  - 20% for OpenMP program
  - 20% for hybrid MPI-OpenMP program
- 40% – Quality of paper, presentation, and clarity of results/explanations
- 20% – Achieving acceptable speedups